Универзитет у Бањој Луци
Електротехнички факултет
Катедра за аутоматику

ДИГИТАЛНИ СИСТЕМИ УПРАВЉАЊА

Извјештај - лабораторијска вјежба 1

Студент: Ненад Кајкут

Број индекса: 1313/17

Бања Лука, 2021.

Израда задатака

Задатак 1. - Фреквенцијско преклапање (*aliasing*)

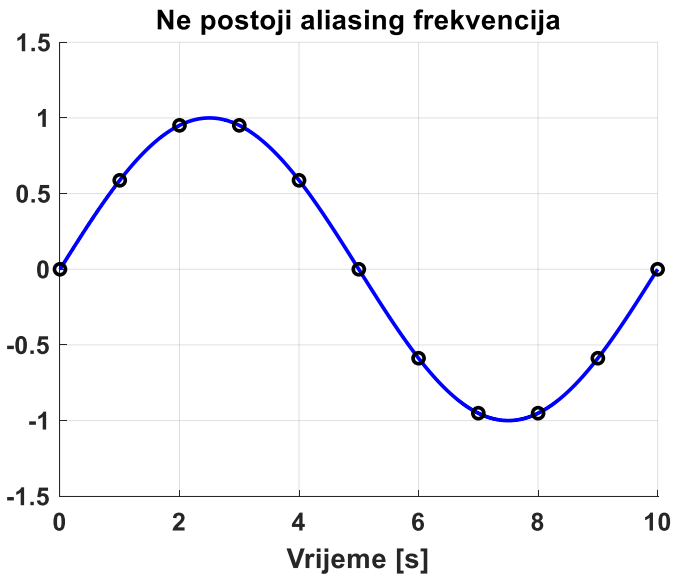
Континуални сигнал који има Фуријеову трансформацију једнаку нули изван интервала $(-\omega_0, \omega_0)$ биће одређен на јединствен начин својим вриједностима у еквидистантним тачкама ако је фреквенција одабирања $\omega_s > 2\omega_0$. Ако би фреквенција одабирања (ω_s) била таква да не задовољава услов $\omega_s > 2\omega_0$, тада се, након дискретизације сигнала, може десити да се одређена фреквенција не би могла разликовати од својих алиаса. Ова појава се дешава ако за сигнал кружне учестаности ω_1 и учестаности одабирања ω_s вриједи услов $\omega_1 > \frac{\omega_s}{2}$. Тада је основни алиас за фреквенцију $\omega_1 > \frac{\omega_s}{2}$ дат изразом:

$$\omega = \left|\left(\omega_1 + \frac{\omega_s}{2}\right) \cdot mod(\omega_s) - \frac{\omega_s}{2}\right|$$

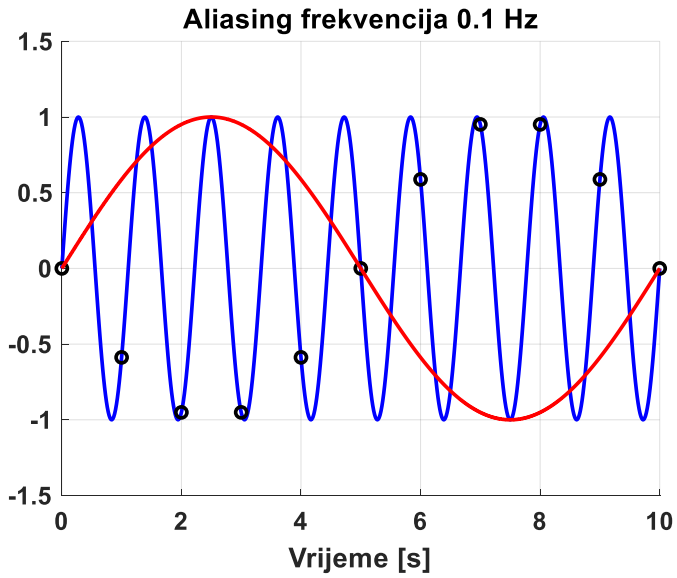
У скрипти програмског пакета *Matlab* у наставку текста долазиће до промјене вриједности времена приказивања сигнала (t_{max}) у зависности од избора фреквенције у појединим примјерима. Такође биће извршена промјена сигнала тако да нам се приказује сигнал синусног облика од првобитно заданог сигнала који је косинусног облика.

Примјере из књиге добијамо ако у скрипти програмског пакета *Matlab* промјенљивој f додијелимо вриједности $f = 0.1[Hz]$ и $f = 0.9[Hz]$ а промјенљивој f_s додијелимо вриједност $f_s = 1[Hz]$.

Након извршавања скрипте добијено је следеће:



Слика 1.1 – Фреквенција континуалног сигнала $f = 0.1[Hz]$, фреквенција одабирања $f_s = 0.1[Hz]$



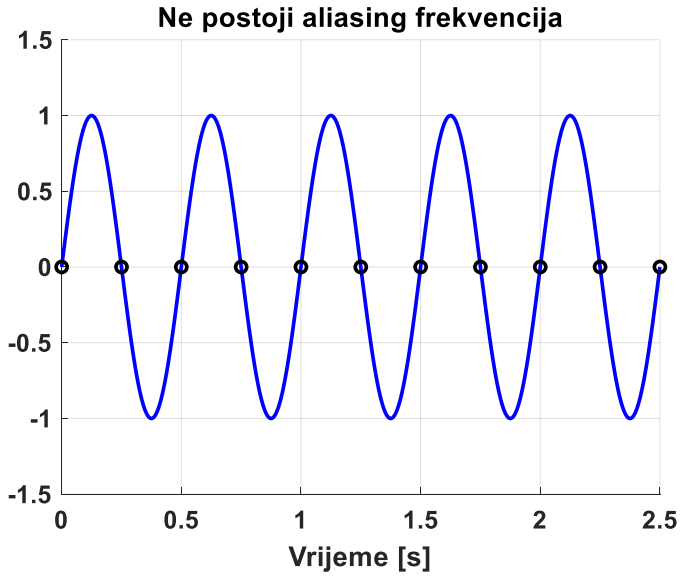
Слика 1.2 – Фреквенција континуалног сигнала $f = 0.9[Hz]$*, фреквенција одабирања* $f_s = 0.1[Hz]$

Са слике 1.1 видимо да не долази до појаве *aliasing*-а, јер је $f_s > 2f$, док са слике 1.2 видимо да долази до појаве *aliasing*-а, јер је $f_s < 2f$. На основу наведених примјера можемо закључити да се појава *aliasing*-а може избјећи у случају када континуални сигнал одабирамо са бар два пута већом фреквенцијом ($f_s > 2f$).

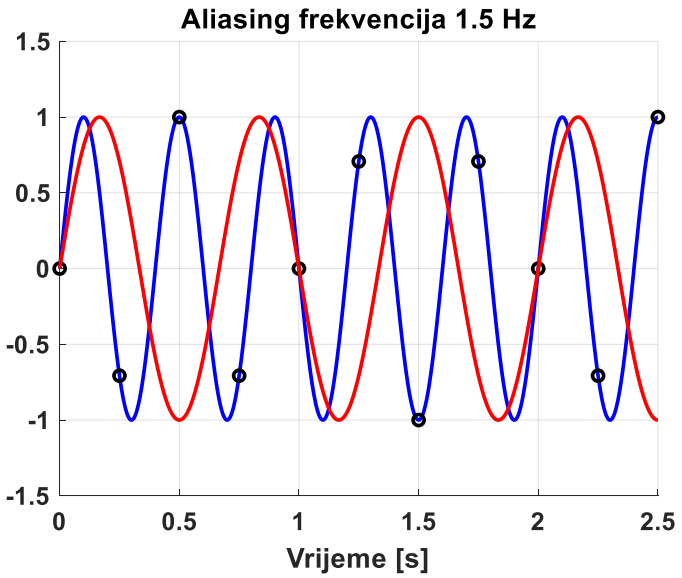
Горе наведене закључке ћемо показати још са по два примјера гдје долази до појаве *aliasing*-а и гдје не долази до појаве *aliasing*-а.

Први примјер

За фреквенцију континуалног сигнала, у скрипти програмског пакета *Matlab* промјенљивој f додијељујемо вриједности $f = 2[Hz]$ и $f = 2.5[Hz]$ а промјенљивој f_s додијељујемо вриједност $f_s = 4[Hz]$.



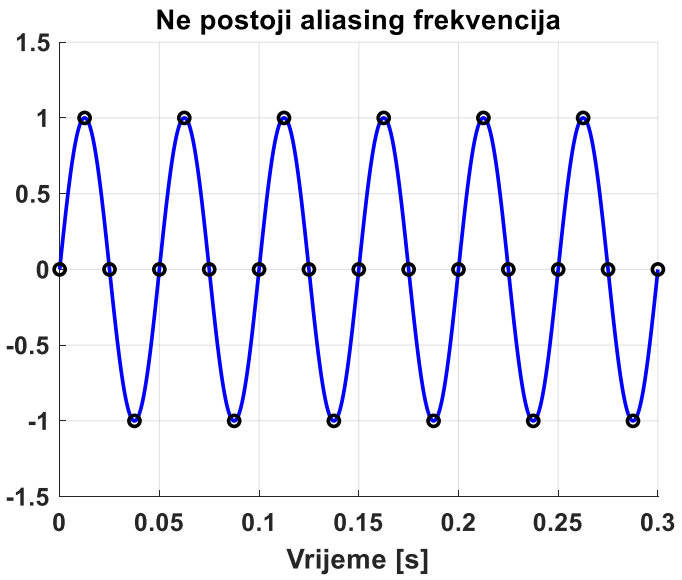
Слика 1.3 – Фреквенција континуалног сигнала $f = 2[Hz]$*, фреквенција одабирања* $f_s = 4[Hz]$



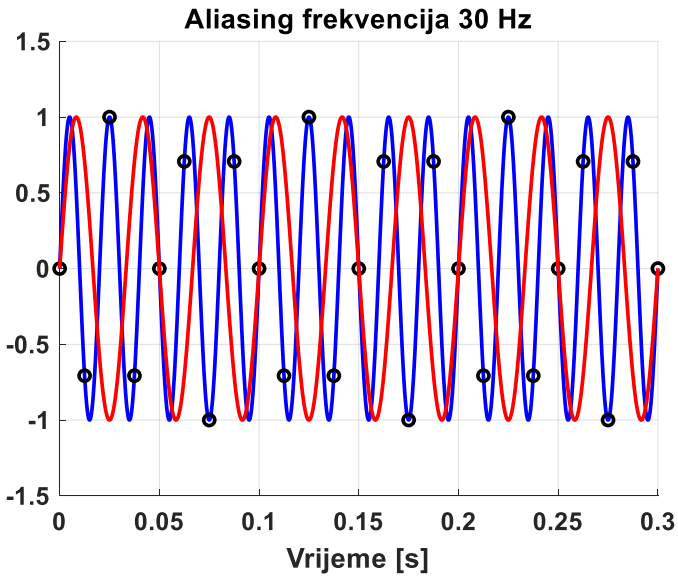
Слика 1.4 – Фреквенција континуалног сигнала $f = 2.5[Hz]$, *фреквенција одабирања* $f_s = 4[Hz]$

Други примјер

За фреквенцију континуалног сигнала, у скрипти програмског пакета *Matlab* промјенљивој f додијељујемо вриједности $f = 20[Hz]$ и $f = 50[Hz]$ а промјенљивој f_s додијељујемо вриједност $f_s = 80[Hz]$.



Слика 1.5 – Фреквенција континуалног сигнала $f = 20[Hz]$, *фреквенција одабирања* $f_s = 80[Hz]$

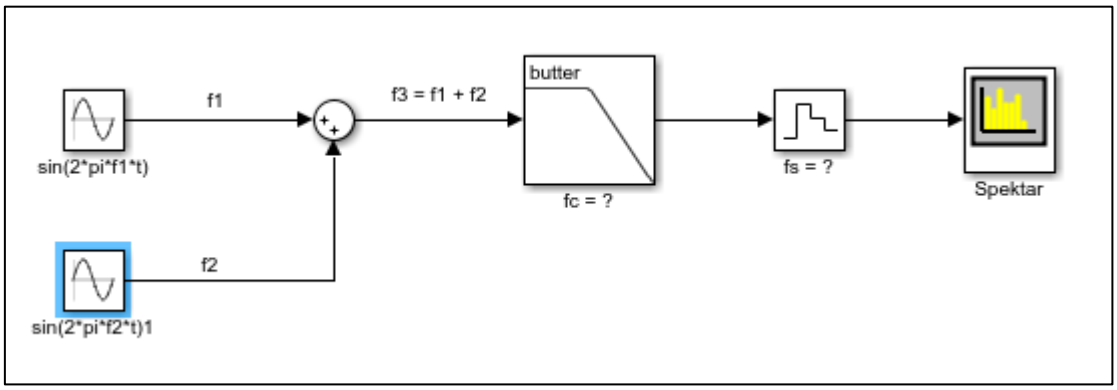


Слика 1.6 – Фреквенција континуалног сигнала $f = 50[Hz]$, *фреквенција одабирања* $f_s = 80[Hz]$

Са слика 1.3 и 1.5 видимо да не долази до појаве *aliasing*-а, јер је $f_s > 2f$, док са слика 1.4 и 1.6 видимо да долази до појаве *aliasing*-а, јер је $f_s < 2f$. На основу наведених примјера можемо закључити да се појава *aliasing*-а може избјећи у случају када континуални сигнал одабирамо са бар два пута већом фреквенцијом ($f_s > 2f$).

Задатак 2. - Предфилтри (*antialiasing* филтри)

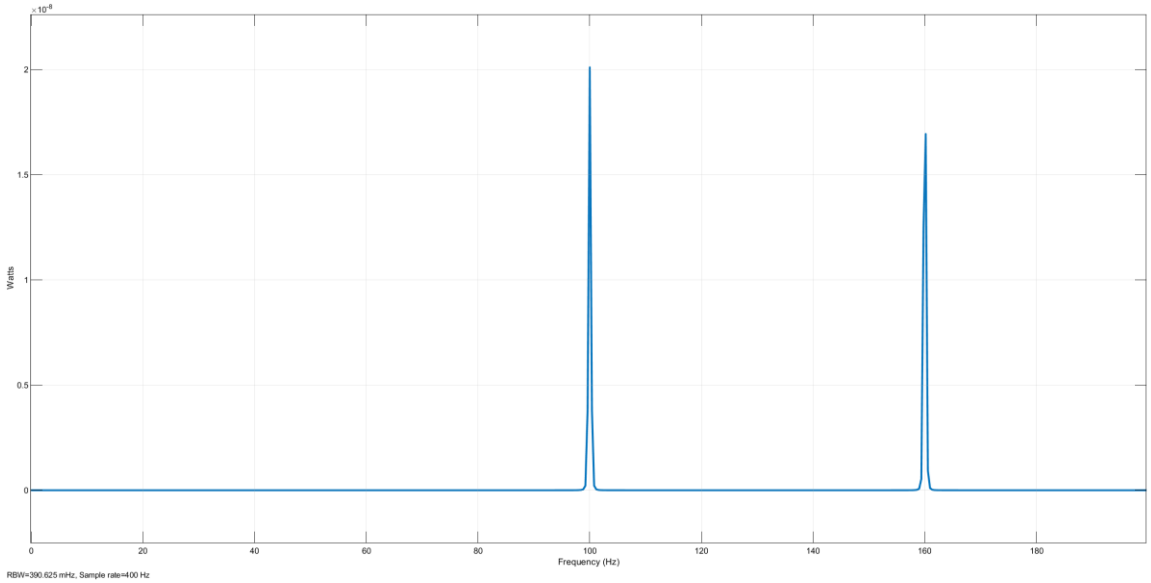
У реалним ситуацијама појављују се проблеми због тога што сигнали који се користе у дигиталној обради често имају спектар који није једнак нули за фреквенције које су веће од Никвистове фреквенције, односно за које нису задовољени услови теореме одабирања. Због тога се у највећем броју случајева мора користити филтрирање сигнала прије A/D конверзије. Као предфилтри најчешће се користе аналогни филтри типа пропусника ниских фреквенција и реда који се бира у складу са захтијеваним слабљењем на Никвистовој фреквенцији и дозвољеним транспортним кашњењем које овакав филтар уноси у систем управљања.



Слика 2.1 – Simulink модел који је коришћен приликом анализе

Да би се сигнал f_3 пропустио у цјелости, потребно је да гранична фреквенција филтра буде већа од највеће фреквенције у спектру. Пропуштени сигнал је потребно одабирати са бар два пута већом фреквенцијом од максималне фреквенције спектра пропуштеног сигнала. На основу наведеног, бирамо следеће вриједности граничне фреквенције $f_c = 200[Hz]$ и фреквенције одабирања $f_s = 400[Hz]$.

Илустрација претходно наведеног приказана је на слици 2.2.

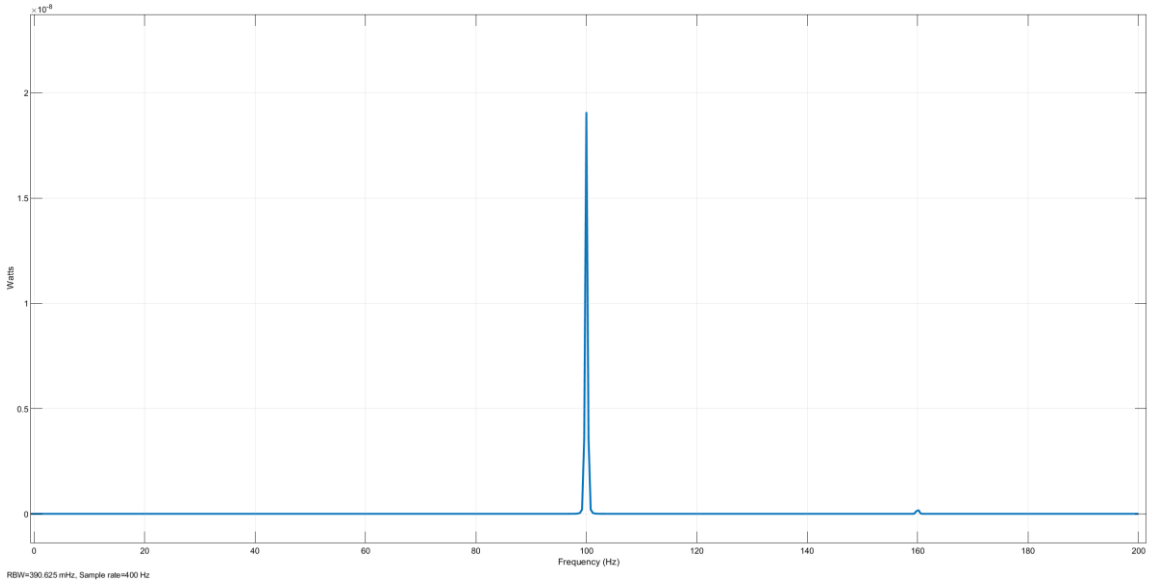


Слика 2.2 – Spectrum Analzyer: $f_1 = 100[Hz]$, $f_2 = 160[Hz]$, $f_c = 200[Hz]$, $f_s = 400[Hz]$

Ради лакшег очитања графика и приказа постојећих компоненти у спектру (слике 2.2), потребно је *y*-осу изразити у *Watts* на следећи начин: након што се симулација покрену отвара нам се прозор *Spectrum Analyzer* → *View* → *Spectrum Settings* → *Trace options* и у пољу *Unit* одаберемо *Watts.* Са слике 2.2 видимо да филтар пропушта сигнале чије су фреквенције $f_1 = 100[Hz]$ и $f_2 = 160[Hz]$, јер су оне мање од граничне фреквенције филтра која износи $f_c = 200[Hz]$.

Да би се пропустио само сигнал чија је фреквенција $f_1 = 100[Hz]$ потребно је да гранична фреквенција филтра буде већа од *100*[*Hz*], а мања од *160*[*Hz*]. На основу наведеног одабрана је вриједност граничне фреквенције $f_c = 120[Hz]$ и фреквенција одабирања $f_s = 400[Hz]$.

Илустрација претходно наведеног приказана је на слици 2.3.



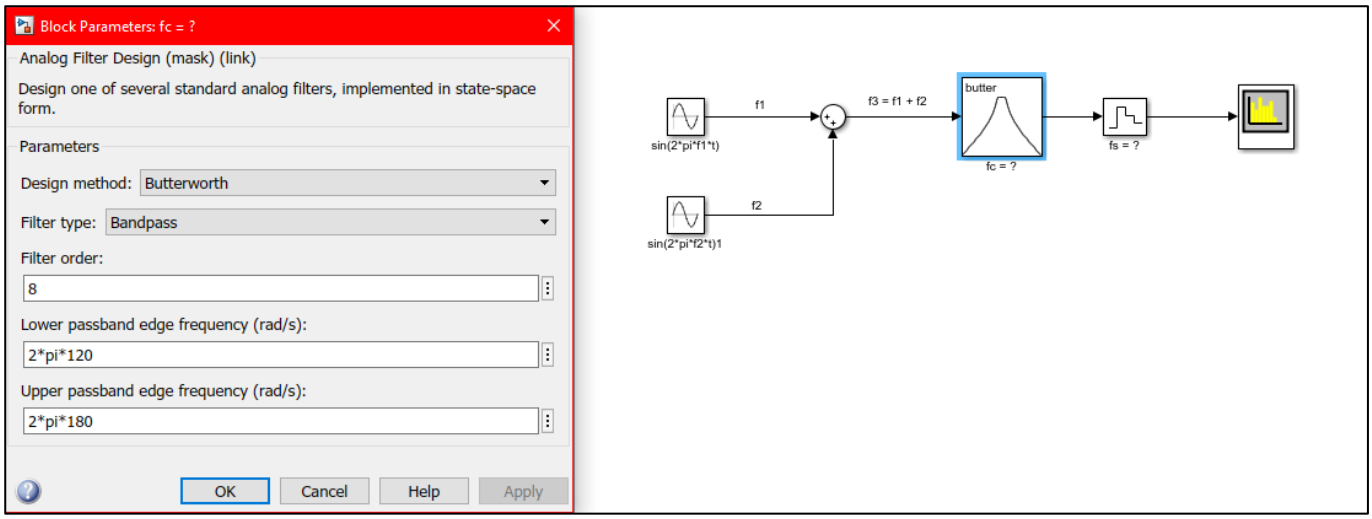
Слика 2.3 – Spectrum Analzyer: $f_1 = 100[Hz]$, $f_c = 120[Hz]$, $f_s = 400[Hz]$

Са слике 2.3 видимо да је спектар различит од нуле само на фреквенцији $f_1 = 100[Hz]$ па на основу тога закључујемо да је пропуштен само сигнал са фреквенцијом $f_1 = 100[Hz]$.

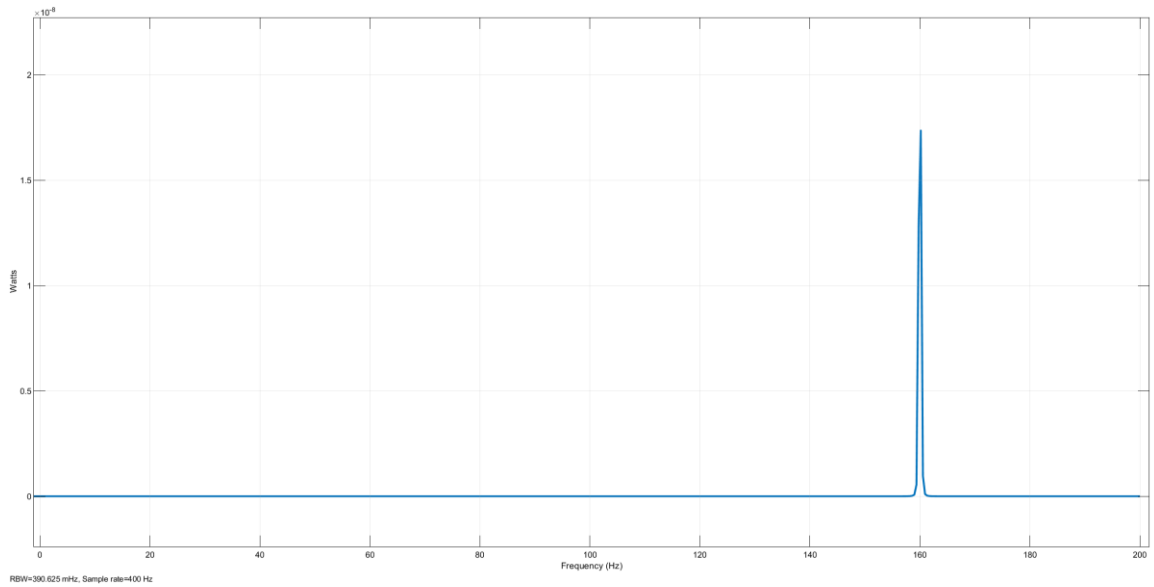
У претходним примјерима кориштен је нископропусни филтар који елиминише компоненте фреквенције које су веће од његове граничне фреквенције, а пропушта оне компоненте фреквенције које се налазе између нуле и његове граничне фреквенције.

Да би се пропустио само сигнал чија је фреквенција $f_2 = 160[Hz]$ потребно је одабрати *Bandpass* тип филтра. То ћемо урадити тако што у оквиру блока *Analog Filter Design* у пољу *Filter type* одаберемо *Bandpass.* У том случају потребно је одабрати двије граничне фреквенције, при чему једна мора бити између фреквенције сигнала $f_1 = 100[Hz]$ и $f_2 = 160[Hz]$, а друга мора бити већа од фреквенције сигнала $f_2 = 160[Hz]$. На основу наведеног одабране су вриједности граничних фреквенција $f_{c1} = 120[Hz]$ и $f_{c2} = 180[Hz]$ и фреквенција одабирања $f_s = 400[Hz]$.

Илустрација претходно наведеног приказана је на слици 2.4.



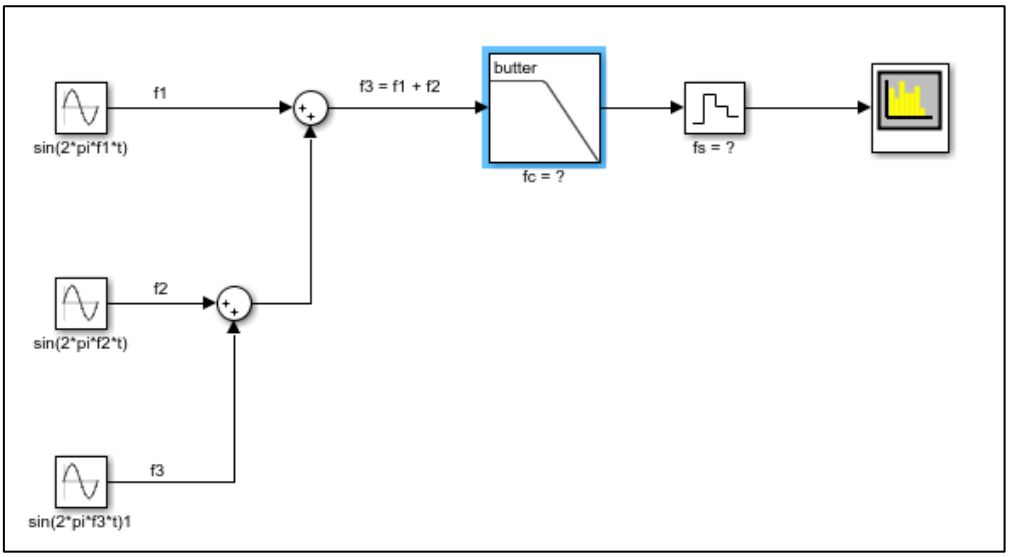
Слика 2.4 – Simulink модел који је коришћен приликом анализе са Bandpass филтром



Слика 2.5 – Spectrum Analzyer: $f_2 = 160[Hz]$, $f_{c1} = 120[Hz]$, $f_{c2} = 180[Hz]$, $f_s = 400[Hz]$

На слици 2.5 можемо видјети да спектар садржи само једну компоненту на фреквенцији *160*$[Hz]$, што значи да је успјешно пропуштен само сигнал са фреквенцијом $f_2 = 160[Hz]$.

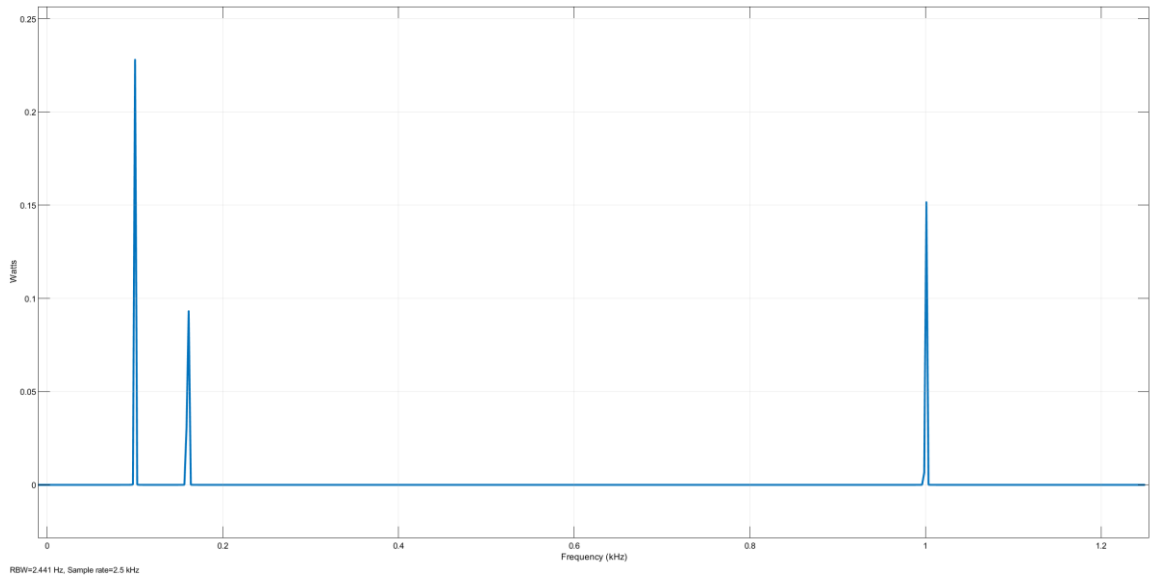
Ако се на један улазни сигнал, нпр. сигнал фреквенције $f_2 = 160[Hz]$ дода и шум (високофреквенцијска компонента) *Simulink* модел ће изгледати као што је приказано на слици 2.6.



Слика 2.6 – Simulink модел који је коришћен приликом анализе када се на један улазни сигнал дода шум

Нека се шум појавио на фреквенцији *1000*[*Hz*]. Тада је потребно изабрати одговарајућу граничну фреквенцију филтра f_c и фреквенцију одабирања f_s да би се на *Spectrum Analyzer*-у добио изворни сигнал. То значи да се спектар мора састојати из три компоненте које се налазе на фреквенцијама *100*[*Hz*], *160*[*Hz*] и *1000*[*Hz*]. Да би филтар пропустио све компоненте сигнала, његова гранична фреквенција мора бити већа од *1000*[*Hz*]. На основу наведеног, бирамо да је вриједност граничне фреквенције $f_c = 1200[Hz]$ и фреквенције одабирања $f_s = 2500[Hz]$.

Илустрација претходно наведеног приказана је на слици 2.7.

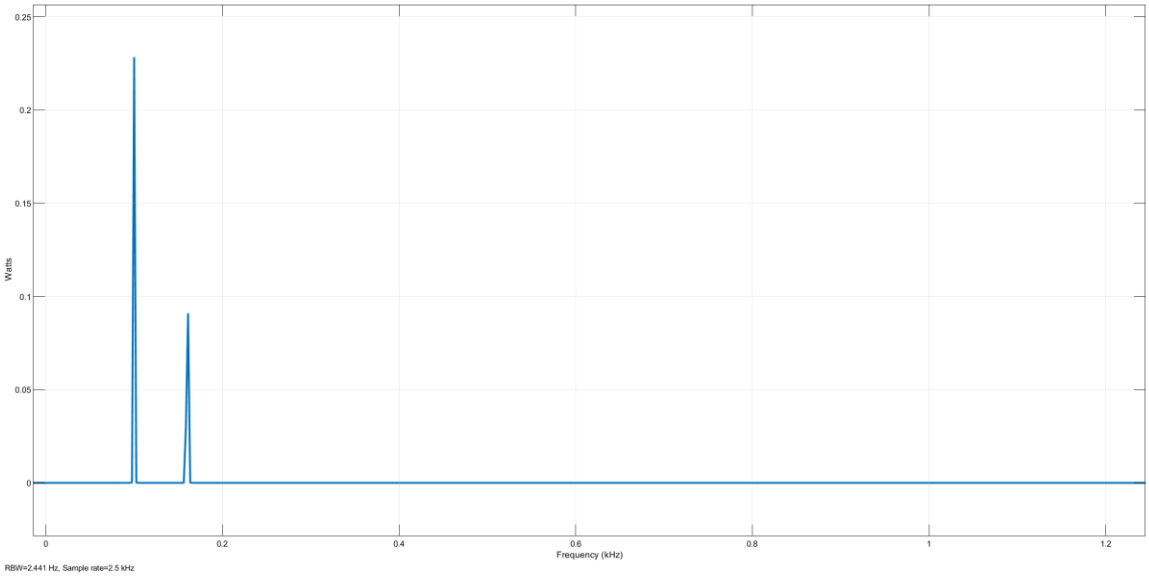


Слика 2.7 – Spectrum Analzyer: $f_1 = 100[Hz]$, $f_2 = 160[Hz]$, $f_3 = 1000[Hz]$ $f_c = 1200[Hz]$, $f_s = 2500[Hz]$

Ако анализатор спектра не треба да види шум, онда га је потребно уклонити помоћу филтра. У том случају потребно је одабрати граничну фреквенцију филтра већу од *160*[*Hz*] ($f_c > 160[Hz]$) и фреквенцију одабирања бар два пута већу од *160*[*Hz*] ($f_s \geq 2 \cdot 160[Hz]$).

На основу претходно наведеног бирамо да нам је гранична фреквенција $f_c = 200[Hz]$ те ћемо на излазу добити спектар сигнала без високофреквентне компоненте која се појављује на *1000*[*Hz*].

Илустрација претходно наведеног приказана је на слици 2.8.



Слика 2.8 – Spectrum Analzyer: $f_1 = 100[Hz]$, $f_2 = 160[Hz]$, $f_c = 200[Hz]$, $f_s = 2500[Hz]$

На основу резултата који су приказани на сликама 2.2 и 2.8 закључујемо да након елиминације шума помоћу филтра, сигнал можемо одабирати са фреквенцијом одабирања $f_s = 200[Hz]$ умјесто са фреквенцијом одабирања $f_s = 2500[Hz]$.

Задатак 3. - Z, Z^{-1}, Z_m трансформација

У програмском пакету *Matlab* илустроваћемо примјену функција *„ilaplace“, „ztrans“, „iztrans“*.

```
>> syms s t z
>> F_s(s) = 1/(s*(s+1))

F_s(s) =

1/(s*(s + 1))

>> f_t(t) = ilaplace(F_s)

f_t(t) =

1 - exp(-t)

>> F_z(z) = ztrans(f_t)

F_z(z) =

z/(z - 1) - z/(z - exp(-1))

>> f_t2(t) = iztrans(F_z)

f_t2(t) =

exp(-1)*(exp(1)*kroneckerDelta(n, 0) - exp(-1)^n*exp(1)) - kroneckerDelta(n, 0) + 1
```

Слика 3.1 – Примјер примјене функција „ilaplace“, „ztrans“, „iztrans“

У *Matlab Help*-у може се наћи објашњење за функцију *kroneckerDelta*. Прототип ове функције је ***kroneckerDelta(m, n)***. Ова функција враћа вриједност 1 ако ***m*** и ***n*** имају исте вриједности а 0 ако ***m*** и ***n*** имају различите вриједности.

У наставку ћемо показати како се врши дискретизација система описаног моделом у простору стања у програмском пакету *Matlab*. Ово ћемо илустровати на једном од примјера који су урађени на аудиторним вјежбама. *Matlab* код који илуструје овај примјер приказан је на слици 3.2.

```
>> A = [0 1; -0.5 -1.5];
>> B = [0; 1];
>> C = [2 1];
>> D = 0;
>> Sys_c = ss(A, B, C, D)

Sys_c =

  A =
          x1    x2
   x1      0     1
   x2   -0.5  -1.5

  B =
       u1
   x1   0
   x2   1

  C =
       x1  x2
   y1   2   1

  D =
       u1
   y1   0

Continuous-time state-space model.

>> T = 0.5;
>> Sys_d = c2d(Sys_c, T)
```

Слика 3.2 – Matlab код који илуструје како се врши дискретизација система описаног моделом у простору стања

Рјешење задатка у *Matlab*-у, тј. Дискретни модел у простору стања приказан је на слици 3.3.

```
Sys_d =

  A =
             x1        x2
   x1    0.9511    0.3445
   x2   -0.1723    0.4343

  B =
             u1
   x1   0.09786
   x2    0.3445

  C =
       x1  x2
   y1   2   1

  D =
       u1
   y1   0

Sample time: 0.5 seconds
Discrete-time state-space model.
```

Слика 3.3 – Дискретни модел система у простору стања

Ако упоредимо дискретни модел система у простору стања који смо добили у програмском пакету *Matlab* са моделом који смо добили на аудиторним вјежбама, можемо закључити да су резултати идентични.